KHOJO

Artificial Intelligence (AI) Powered Missing Person Finder with Facial Aging Technology

Functional Requirements Specification (FRS)

Version 1.1 — Detailed Edition

Project	KHOJO
Hackathon	Amnex Hackathon 2026
Use Case	UC34 — Missing Person Finder
Domain	Social Impact
Version	1.1 (Detailed)
Date	24 April 2026
Status	Final

Team KHOJO | Amnex Hackathon 2026

1. Introduction

1.1 Purpose

This Functional Requirements Specification (FRS) document describes the complete functional behaviour, user interactions, technical architecture, system design, and acceptance criteria of KHOJO — an Artificial Intelligence (AI) powered mobile and web solution designed to locate missing persons through facial aging prediction and age-invariant face recognition. This version expands every section with architectural detail, data flows, component diagrams, and domain-specific implementation guidance.

1.2 Intended Audience

- Hackathon judges and evaluation panel
- Project developers, testers, and architects
- Domain experts from law enforcement and non-governmental organisations (NGOs)
- Government officers responsible for missing person cases
- Future open-source contributors

1.3 Document Conventions

- "Must", "shall", and "required" denote mandatory requirements.
- "Should" and "recommended" denote preferred but non-mandatory requirements.
- All technical acronyms are expanded at first occurrence in each section.
- Requirement identifiers follow the pattern FR-X.Y, NFR-N, AC-N, AR-N.

2. Problem Statement

India records more than one hundred thousand (100,000+) missing person cases every year. A significant proportion of these are children who go missing at an early age and, when eventually found or sighted years later, are unrecognisable because their facial features have changed considerably due to aging. Families, shelter homes, and law-enforcement officers lack a reliable tool to bridge the visual gap between the last known photograph and the present-day appearance of the missing individual.

> **Core Challenge:** A ten-year-old child missing in 2009 would appear as a 27-year-old adult in 2026 — a face that no photograph in the case file resembles.

KHOJO addresses this problem by applying Artificial Intelligence to predict how a missing person is likely to appear today, based on one or more older photographs, and by automatically matching this predicted face against a growing database of sighted and registered individuals.

2.1 Scale of Problem — India Statistics

Metric	Figure	Source
Missing persons/year	1,00,000+	NCRB Annual Report
Children as % of missing	~45%	Missing Link Trust
Cases solved within 1 year	~30%	NCRB
Average duration of search	3–7 years	Field survey

3. Scope of the Solution

The solution is delivered as three tightly integrated components:

- Mobile Application — used by community members, family members, and field workers (React Native, Android + iOS).
- Backend Application Programming Interface (API) — the AI processing layer performing face detection, embedding, aging, recognition, scoring, and alert routing (Python FastAPI).
- Administrator Console — a role-based web interface for government officers and senior police (React web, embedded in mobile build).

4. System Architecture

4.1 High-Level Architecture

Architecture Principle: All computer vision processing runs locally on-premise. No biometric data is ever transmitted to external services. Only optional text summaries use an external API (OpenAI GPT-4o).

KHOJO — System Layer Diagram

[PRESENTATION LAYER]
React Native Mobile App (iOS + Android) | React Web Admin Console

[API GATEWAY]
FastAPI — REST Endpoints | JWT Authentication | Rate Limiter | Circuit Breaker

[AI PROCESSING ENGINE] (fully local)
InsightFace (Detection + Landmarking) → ArcFace (Embedding) → Aging Trajectory Module → SAM/HRFAE GAN (Age Progression) → ArcFace Cosine Similarity (Recognition) → Confidence Scorer → Alert Router

[DATA LAYER]
PostgreSQL (Cases, Users, Audit Log) | Redis (Job Queue, Session) | S3-compatible Object Store (Images)

[OPTIONAL EXTERNAL]
OpenAI GPT-4o API (Case Summaries + Alert Messages only) | GeoPy Geocoding

4.2 Component Architecture

Component	Technology	Responsibility	Scalability
Mobile App	React Native 0.73+	All user-facing flows; camera access; OTP auth	Horizontal — stateless clients
API Server	Python 3.11 + FastAPI	Business logic, AI pipeline orchestration	Docker replicas behind Nginx
Task Queue	Celery + Redis	Async face aging jobs (GPU-bound)	Multiple Celery workers per GPU

Face Detection	InsightFace 0.7	Detect faces, extract 68 landmarks	CPU or GPU
Face Embedding	ArcFace (ResNet50)	512-d cosine-space face fingerprint	GPU batch inference
Age Progression	SAM / HRFAE GAN	Generate aged photograph	Single GPU worker
Face DB	pgvector (PostgreSQL ext.)	Vector index for ANN similarity search	Index sharding by state
Geocoding	GeoPy + Nominatim	Resolve addresses to lat/lon; clustering	Cached results in Redis
Audit Store	PostgreSQL (append-only)	Tamper-evident decision log	Partitioned by month
Object Store	MinIO (S3-compatible)	Original + aged photos, immutable	Multi-node replication
Admin Console	React 18 (embedded)	Admin five-tab interface	Served via FastAPI static

4.3 AI Pipeline — Detailed Data Flow

Step	Operation	Detail	Technology	Output
1	**Face Detection**	RetinaFace detector from InsightFace locates the primary face bounding box. 68 facial landmark key points (eyes, nose, mouth, jaw, brows) are extracted and normalised to 112×112px.	InsightFace / RetinaFace	Bounding box + 68-point landmark map
2	**Face Embedding**	Aligned face patch fed through ArcFace (ResNet-50 backbone, trained on MS1MV3). Output is a 512-dimensional L2-normalised vector — the 'face fingerprint' that is age-invariant by design.	ArcFace / InsightFace	512-d embedding vector
3	**Aging Trajectory**	For each photo uploaded (minimum 2), the embedding is tagged with the known age. A linear interpolation over the embedding space between the two known ages yields the per-dimension aging rate vector Δv.	Custom Python module	Per-dimension aging rate Δv
4	**Age Progression**	Δv is used to prime the GAN's latent code. SAM (Style-based Age Manipulation) maps the input image to a target age using the StyleGAN2 latent space. HRFAE is used as fallback for better high-resolution output.	SAM / HRFAE GAN (PyTorch)	Aged RGB image at predicted age
5	**Target Embedding**	ArcFace extracts a fresh 512-d embedding from the GAN-generated aged image.	ArcFace	Target embedding e_target

6	**DB Search**	e_target is compared against all registered sighting embeddings using cosine similarity. pgvector's IVFFlat index returns top-K candidates in sub-second time.	pgvector (PostgreSQL)	Ranked list of (candidate_id, score)
7	**Confidence Score**	cosine_similarity(e_target, e_candidate) mapped to [0.0, 1.0] using sigmoid calibration. Threshold routing: ≥0.80 → High, 0.60–0.80 → Medium, <0.60 → Low.	Python + NumPy	Score in [0,1] + tier label
8	**Alert Routing**	High: immediate push notification to assigned field worker. Medium: queued for human review. Low: marked inconclusive.	FastAPI + Firebase FCM	Push alert or review queue entry
9	**Summary (optional)**	GPT-4o called with structured case data + system prompt version ID. Output: investigator summary + community alert text.	OpenAI GPT-4o API	Plain-language summary strings
10	**Audit Log**	All inputs, outputs, model versions, confidence scores, and timestamps written atomically to append-only PostgreSQL table with SHA-256 chained checksum.	PostgreSQL	Tamper-evident audit record

4.4 Database Schema — Core Tables

Table	Key Columns	Type	Notes
cases	id, khj_id, name, dob, missing_year, last_location, status	Core	khj_id = KHJ-YYYY-XXXXX. status ∈ {active, review, matched, closed}
photos	id, case_id, s3_key, age_at_capture, embedding (vector 512)	Core	pgvector column enables ANN search. Immutable after upload.
sightings	id, reporter_id, location, photo_s3_key, embedding (vector 512), reported_at	Core	Uploaded by field workers or community; compared against case embeddings.
matches	id, case_id, sighting_id, score, tier, status, reviewed_by	Workflow	status ∈ {pending, confirmed, rejected}. reviewed_by FK to users.
users	id, name, phone_hash, role, zone,	Auth	Passwords never stored. OTP via TOTP. gov_id hashed with bcrypt.

	gov_id_hash, otp_secret		
audit_log	id, ts, action, case_id, model_version, input_hash, output_hash, confidence, prev_checksum, checksum	Compliance	Append-only. checksum = SHA-256(row_data || prev_checksum). prev_checksum chains entries.
model_versions	id, name, deployed_at, val_accuracy, training_pool_size	AI Ops	Admin console reads this for display. Fine-tune increments version.
training_pool	id, case_id, predicted_s3_key, actual_s3_key, error_vector (json), created_at	Continuous Learning	Populated on 'Not a Match' submissions. Fine-tune triggered at count=50.

5. Users and Roles

Actor	Description	Key Permissions
Community Member / Family	Ordinary citizen with a missing relative or witness to an unknown individual.	Register case, upload photos, view match results and status updates.
Field Worker	NGO representative or constable operating in the field.	Receive alerts, physically verify matches, upload sighting photos, submit Confirm / Not-a-Match feedback.
Administrator	Government officer or senior police official.	All field-worker permissions + approve/reject cases, tune AI thresholds, manage field workers, export audit logs.
System (AI, Automated)	Fully automated — no human involvement.	Execute AI pipeline, compute confidence, dispatch alerts, log audit entries, trigger fine-tuning.

6. Functional Requirements

6.1 Registration and Authentication

FR#	Requirement
FR-1.1	The mobile application must allow every user to register using name, phone number, and location, and to select their role (Family Member, Community Member, Field Worker, or Administrator).
FR-1.2	Authentication must be performed through a One-Time Password (OTP) sent to the registered mobile number. Administrators must additionally authenticate using a police/government identification number (two-factor authentication, 2FA).
FR-1.3	The mobile application must confirm successful submission of any case or verification with a clear on-screen acknowledgement and a unique reference identifier in the format KHJ-YYYY-XXXXX.

FR-1.4	Input validation must reject malformed or out-of-range data (e.g., future years, invalid ages, unreadable images) with a plain-language error message in the user's language.
FR-1.5	OTP codes must expire after five (5) minutes and must be rate-limited to three (3) requests per phone number per fifteen-minute window to prevent abuse.
FR-1.6	Session tokens (JWT) must expire after twenty-four (24) hours. Refresh tokens are valid for thirty (30) days. On device loss, the user can revoke all active tokens via OTP re-authentication.

6.2 Missing Person Case Registration

FR#	Requirement
FR-2.1	The user must be able to enter: missing person's full name, year of disappearance, age at disappearance, last known location (city/area), and relationship to the reporter.
FR-2.2	The system must automatically compute and display Predicted Present-Day Age = Age at Disappearance + (Current Year − Year of Disappearance). This field must be read-only and re-computed if inputs are changed.
FR-2.3	The user must upload a minimum of two (2) photographs, each tagged with the person's age at the time the photograph was taken. Additional photographs may be uploaded to improve accuracy.
FR-2.4	The system must validate each uploaded photograph: minimum resolution 224×224 px, maximum file size 10 MB, formats JPEG/PNG only, and must detect exactly one face using InsightFace before accepting the image.
FR-2.5	The user may optionally provide identifying marks (scars, moles, birthmarks, tattoos) in a structured free-text field to assist manual verification.
FR-2.6	On successful submission, the system must return a Case Identifier in format KHJ-YYYY-XXXXX and send an SMS confirmation to the registered phone number.

6.3 Core AI Processing

FR#	Requirement
FR-3.1	Face Detection: InsightFace (RetinaFace) must detect the primary face bounding box and extract 68 facial landmark key points from each photograph before any further processing.
FR-3.2	Face Embedding: ArcFace (ResNet-50, trained on MS1MV3) must convert each detected face into a 512-dimensional L2-normalised embedding vector. Embeddings must be stored in the pgvector column of the photos table.
FR-3.3	Aging Trajectory Calculation: A custom module must compute the per-dimension aging rate vector Δv by linear interpolation between the embeddings of the two (or more) tagged photographs across their age gap.
FR-3.4	Face Aging Generation: The system must use a SAM (Style-based Age Manipulation) or HRFAE (High-Resolution Face Age Editing) GAN to synthesise a realistic aged photograph at the predicted present-day age. The GAN must run on GPU; a CPU fallback mode is permitted for demonstration purposes only.
FR-3.5	Age-Invariant Recognition: The aged embedding must be compared against every embedding in the sightings database using ArcFace cosine similarity via pgvector's IVFFlat index. Top-K (default K=10) candidates must be returned.
FR-3.6	Confidence Scoring: Every candidate match must produce a confidence score in the range 0.0 to 1.0 derived from sigmoid-calibrated cosine similarity.
FR-3.7	Alert Routing: Based on configurable thresholds, the system must: (a) ≥80% — dispatch an immediate push notification to the assigned field worker; (b) 60–80% — enqueue the case for mandatory human review; (c) <60% — mark the case as inconclusive and store for manual review.

FR-3.8	Geo-Spatial Clustering: GeoPy must geocode all sighting locations. When three or more sightings or cases cluster within a configurable radius (default 5 km), a geo-alert must be raised to the administrator dashboard.
FR-3.9	Summary Generation (optional): When enabled, GPT-4o must generate: (a) a concise investigator case summary; (b) a community-facing alert message. Every GPT-4o invocation must include a deterministic system-prompt version identifier so that responses are reproducible and auditable.

6.4 Output and User Experience

FR#	Requirement
FR-4.1	The mobile application must present every AI-generated result on a clearly labelled screen showing: (a) original photographs; (b) AI-generated aged photograph; (c) best candidate match photograph (if any); all displayed side-by-side with labels.
FR-4.2	Every AI-generated decision must display: (a) a numerical confidence score; (b) a colour-coded confidence bar (green ≥80%, amber 60–80%, red <60%); (c) a plain-language explanation of how the result was derived.
FR-4.3	The result screen must classify the outcome into the three tiers defined in FR-3.7 and display the appropriate system action in human-readable text.
FR-4.4	A plain-language disclaimer must always appear: "This is an estimate produced by Artificial Intelligence (AI). Please have the result verified by a certified officer before acting upon it."
FR-4.5	All user-facing text must expand AI, ML, API, GAN, OTP, NGO, and every other domain-specific acronym at its first occurrence on each screen.

6.5 Field Worker Verification and 'Not a Match' Feedback Loop

FR#	Requirement
FR-5.1	Field workers must receive in-app push alerts for every high-confidence or medium-confidence match, showing: predicted aged photograph, candidate sighting photograph, location, confidence score, and tier label.
FR-5.2	After physical verification, the field worker must mark the case as 'Confirm Match' or 'Not a Match'. No other outcome is accepted.
FR-5.3	On 'Confirm Match': the system must automatically notify the registered family member with a message composed by GPT-4o (when enabled) or a standard template (when GPT-4o is disabled). The case status is updated to 'matched'.
FR-5.4	On 'Not a Match': the field worker must mandatorily upload the actual photograph of the individual encountered. The system must refuse to accept the submission without this photograph.
FR-5.5	The backend must then: (1) run InsightFace/ArcFace feature comparison between AI prediction and uploaded real photograph; (2) compute a per-feature error vector covering nose shape, cheekbone structure, jawline, eye spacing, and forehead width; (3) append the error vector to the training_pool table; (4) trigger a model fine-tune cycle when the pool accumulates 50 examples, advancing model version (v1.0 → v2.0); (5) notify the family that the match was not confirmed and the case remains active; (6) automatically reopen the case.

6.6 Administrator Controls

The administrator console must provide the following five tabs:

Tab 1 — Overview Dashboard

- Live counts: total cases, active searches, matches found, cases pending human review.

- AI confidence distribution chart: three bars — High (≥80%, green), Medium (60–80%, amber), Low (<60%, red).
- Real-time recent activity feed (case submitted, match found, field worker dispatched).
- Geo-heatmap layer showing sighting density across states.

Tab 2 — Case Management

- Filter by status: All / Review Pending / Found / Closed.
- Each case shows: name, Case Identifier, predicted age, match location, confidence score, assigned field worker.
- Administrators can Approve (dispatch field-worker alert) or Reject (close case, notify family).

Tab 3 — Field Worker Management

- List of all field workers with zone assignment, verification count, and personal accuracy score.
- Ability to add, reassign, or remove a field worker.
- Leave-status management with automatic reassignment of open cases.

Tab 4 — AI Settings (No-Code Threshold Tuning)

Setting	Type	Default	Range
Confidence Review Threshold	Slider	60%	40% – 90%
Auto-Alert Threshold	Slider	80%	60% – 99%
Geo-Clustering Radius	Numeric input	5 km	1 – 50 km
Geo-Clustering Minimum Count	Numeric input	3 sightings	2 – 10
GPT-4o Summaries	Toggle	On	On / Off
Geo-Clustering Alerts	Toggle	On	On / Off
Active AI Model Version	Read-only display	v1.0	—

Tab 5 — Audit Log

- Every AI decision recorded: timestamp, model version, action, confidence score, outcome.
- Entries are cryptographically signed (SHA-256 chained checksum) and append-only — edits and deletions are disallowed.
- CSV export available for any selected date range.
- Personally Identifiable Information (PII) is redacted from all entries except where legal retention is required.

7. Non-Functional Requirements

NFR#	Category	Requirement
NFR-1	**Performance**	End-to-end primary-user flow (photo upload → result display) must complete in under five (5) seconds for typical input, excluding network latency. Face-aging module alone must produce output within three (3) seconds.

NFR-2	**Stability**	Backend must tolerate a representative dataset during a fifteen (15) minute demonstration run without crashes, memory leaks, or exceeding the stated hardware budget.
NFR-3	**Graceful Degradation**	When any external API is temporarily unavailable, the flow must degrade gracefully or fail with a user-understandable message. The user must never be left without a response.
NFR-4	**Accessibility**	All user-facing text must expand AI, ML, API, GAN, OTP, and every domain-specific acronym at its first occurrence on each screen.
NFR-5	**Reliability**	Every external API call must implement retry with exponential backoff (up to 3 retries, base delay 500 ms) and a circuit-breaker (open after 5 consecutive failures, half-open after 30 seconds).
NFR-6	**Security & Privacy**	PII must be redacted from all logs except where legal retention is required. Audit entries must be cryptographically signed and tamper-evident. All data in transit must use TLS 1.3. Images stored in object store must be encrypted at rest (AES-256).
NFR-7	**Scalability**	Architecture must support horizontal scaling via Docker Compose replicas. pgvector IVFFlat index must maintain sub-second search at 1,000,000 embeddings.
NFR-8	**Observability**	The backend must emit structured JSON logs to stdout. Health-check endpoints (GET /health) must return service status within 200 ms. Failed containers must restart automatically.

8. Technology Stack

Layer	Technology	Notes
Mobile Application	React Native 0.73+ (Expo)	Single codebase for Android and iOS. Expo Camera for photo capture.
Backend API	Python 3.11 + FastAPI	Async endpoints, Pydantic v2 validation, auto-generated OpenAPI docs.
Task Queue	Celery 5 + Redis 7	GPU-bound face-aging jobs dispatched asynchronously. Flower UI for monitoring.
Face Detection & Landmarks	InsightFace 0.7 (RetinaFace)	~500 MB model. Detects face, extracts 68-point landmarks, normalises to 112×112 px.
Face Recognition	ArcFace ResNet-50 (MS1MV3)	Produces 512-d L2-normalised embeddings. Cosine similarity for matching.
Face Aging GAN	SAM / HRFAE (PyTorch 2.x)	~1–2 GB model. GPU required for production; CPU fallback for demo.
Vector Database	PostgreSQL 15 + pgvector 0.6	IVFFlat index. ANN search in sub-second at 1M embeddings.
Primary Database	PostgreSQL 15	Cases, users, audit log, training pool.
Object Storage	MinIO (S3-compatible)	Immutable image storage. Versioning enabled. AES-256 at rest.
Caching / Session	Redis 7	JWT blacklist, OTP codes, geocode cache, Celery broker.
Geocoding	GeoPy + Nominatim	Local Nominatim instance to avoid rate limits. Cluster detection in Python.

AI Summaries (optional)	OpenAI GPT-4o API	Called only for case summaries. System prompt versioned. Circuit-breaker applied.
Deployment	Docker Compose (docker-compose.prod.yml)	Health checks, automatic restart, GPU device mapping.
Push Notifications	Firebase Cloud Messaging (FCM)	Field-worker alerts. Retry with exponential backoff.
API Gateway	Nginx 1.25	TLS termination, rate limiting, reverse proxy to FastAPI.

9. API Endpoint Reference

Method	Endpoint	Auth	Description
POST	/auth/register	None	Register a new user. Returns JWT + refresh token after OTP verification.
POST	/auth/otp/verify	None	Verify OTP code. Returns short-lived session token.
POST	/cases	JWT (any role)	Create a new missing person case. Returns KHJ case identifier.
POST	/cases/{id}/photos	JWT (case owner)	Upload photographs. Returns photo_id and embedding status.
POST	/cases/{id}/process	JWT (case owner)	Trigger AI pipeline asynchronously. Returns job_id.
GET	/jobs/{job_id}	JWT (case owner)	Poll job status and result (including confidence score and aged image URL).
POST	/sightings	JWT (field worker)	Submit a sighting report with photo. Returns sighting_id.
POST	/matches/{id}/confirm	JWT (field worker)	Confirm a match. Triggers family notification.
POST	/matches/{id}/not-a-match	JWT (field worker)	Submit 'Not a Match' with mandatory photo. Triggers error vector computation.
GET	/admin/dashboard	JWT (admin)	Real-time overview stats.
GET	/admin/cases	JWT (admin)	Paginated case list with filters.
PUT	/admin/settings	JWT (admin)	Update AI thresholds (no code deployment required).
GET	/admin/audit-log	JWT (admin)	Paginated tamper-evident audit log.
GET	/admin/audit-log/export	JWT (admin)	CSV export of audit log for a date range.
GET	/health	None	Service health check. Returns 200 OK with component statuses.

10. Security Architecture

10.1 Authentication & Authorisation

- OTP delivered via SMS (Twilio or MSG91). Codes are six-digit, valid for 5 minutes, rate-limited to 3 attempts per 15 minutes.
- Administrators require 2FA: OTP + hashed government identification number (bcrypt, cost factor 12).
- JWT tokens (HS256) expire after 24 hours. Refresh tokens (opaque, stored in Redis) expire after 30 days.
- Role-Based Access Control (RBAC): community_member < field_worker < administrator. Each API endpoint declares its minimum required role.

10.2 Data Privacy

- All biometric embeddings and photographs stored server-side only. No biometric data transmitted to OpenAI or any external service.
- Phone numbers stored as bcrypt hashes (cost 12). Only OTP delivery service receives the plain number, immediately.
- Government ID numbers stored as bcrypt hashes. Plain numbers are never persisted.
- Audit log redacts PII fields before writing, unless legal retention applies (configurable per jurisdiction).
- Images stored in MinIO with AES-256 encryption at rest. Access requires signed S3 URL with 1-hour expiry.

10.3 Tamper-Evident Audit Chain

Algorithm: Each audit record stores: checksum = SHA-256(row_id || timestamp || action || confidence || input_hash || output_hash || prev_checksum). The chain is verified by the admin console on every export. Any gap or mismatch is flagged immediately.

11. Continuous Learning — 'Not a Match' Feedback Loop

Stage	Action	Technical Detail
1	**Field worker submits 'Not a Match' + real photo**	Photo validated (face detected, min 224×224 px). Stored in MinIO. Record created in training_pool.
2	**Feature extraction**	ArcFace extracts embeddings from both AI-generated prediction and the actual photo. Per-feature landmarks compared using InsightFace 68-point map.
3	**Error vector computation**	Compute Δ per landmark group: nose shape, cheekbone, jawline, eye spacing, forehead width, chin shape. Store as JSONB in training_pool.error_vector.
4	**Threshold check**	COUNT(training_pool WHERE model_version = current) checked. If ≥ 50, fine-tune is triggered.
5	**Model fine-tuning**	Celery task dispatches fine-tune job on the GAN model using the 50 error vectors as curriculum. Model version incremented (e.g., v1.0 → v2.0). admin_log updated.
6	**Strategic benefit**	Over thousands of 'Not a Match' submissions, the generic global GAN adapts to South Asian facial aging characteristics without requiring a bespoke dataset.

12. Acceptance Criteria

AC#	Criterion	Pass Condition
AC-1	**End-to-end flow**	A new user completes input → AI result in a single session without instructions beyond UI prompts.
AC-2	**Face-aging speed**	Face-aging module produces a measurable, testable output on a representative input within three (3) seconds.
AC-3	**Confidence score display**	Every AI decision returns a visible numerical confidence score and a plain-language explanation.
AC-4	**Fallback path**	A score below the configurable threshold (default 0.60) triggers documented fallback — human review, alternative model, or graceful 'unable to determine' — never an unvetted answer.
AC-5	**Audit log**	Every AI invocation is written to the audit log with timestamp, input hash, output hash, model version, and confidence score.
AC-6	**Stability**	Backend runs a 15-minute demonstration on a representative dataset without crashes, memory leaks, or exceeding hardware budget.
AC-7	**Acronym expansion**	All user-facing text expands AI, ML, API, GAN, OTP, and every domain-specific acronym at first occurrence on each screen.
AC-8	**Graceful error handling**	When any external API is unavailable, the flow degrades gracefully or fails with a user-understandable message.
AC-9	**'Not a Match' feedback loop**	Field-worker 'Not a Match' submission with mandatory photo updates training_pool and triggers fine-tune after 50 samples.
AC-10	**Age formula correctness**	Predicted age = Age at Disappearance + (Current Year − Year of Disappearance), computed correctly for any valid input.
AC-11	**2FA for administrators**	Administrator login requires both OTP and government identification number; either alone is rejected.
AC-12	**Tamper-evident audit**	Audit log export includes SHA-256 chained checksums; any tampered entry is detectable by admin console verification.

13. Risks and Mitigations

Risk	Likelihood	Impact	Mitigation
False positive matches causing distress to families	**Medium**	**High**	Three-tier confidence classification, mandatory field-worker physical verification, plain-language disclaimer on every result screen.
Bias in face-aging GAN towards non-Indian facial features	**High (initially)**	**Medium**	Continuous-learning feedback loop from 'Not a Match' submissions progressively tunes the model to the Indian subcontinent without a bespoke dataset.
External OpenAI API outage	**Low**	**Low**	GPT-4o summaries are optional. Fallback to standard message templates when API is unavailable.
PII misuse or data breach	**Low**	**High**	2FA for admins, PII redaction in logs, AES-256 at rest, TLS 1.3 in transit, append-only tamper-evident audit chain.
GPU unavailability in demonstration environment	**Medium**	**Medium**	CPU fallback mode documented and configured. Performance expectations clearly stated on UI during fallback.

Photo quality insufficient for face detection	Medium	Medium	FR-2.4 validates photo before acceptance. User shown clear error with guidance on retaking the photograph.
Model fine-tuning corrupting production weights	Low	High	Fine-tuning runs on a copy of the model. Production weights replaced only after validation accuracy check passes threshold.

14. Future Enhancements

- Integration with the Crime and Criminal Tracking Network and Systems (CCTNS) database for real-time cross-referencing.
- Multilingual support: Hindi, Gujarati, Marathi, Tamil, Bengali, Telugu, and other Indian languages.
- Edge deployment on field-worker devices for offline verification in low-connectivity zones.
- WhatsApp chatbot for case registration by families who do not use smartphone applications.
- Federated learning across states to improve the model without centralising sensitive biometric data.
- Integration with Closed-Circuit Television (CCTV) feeds at railway stations and bus terminals.
- Aadhaar-linked verification pathway for confirmed identity resolution (subject to regulatory approval).

15. Glossary

Term / Acronym	Definition
AI — Artificial Intelligence	A branch of computer science concerned with building systems that perform tasks typically requiring human intelligence.
API — Application Programming Interface	A defined contract enabling two software components to communicate.
ArcFace	A face-recognition loss function and model producing angular-margin embeddings for highly discriminative recognition.
ANN — Approximate Nearest Neighbour	A class of search algorithm that finds the closest vectors in high-dimensional space efficiently.
CSV — Comma-Separated Values	A plain-text file format for tabular data, used here for audit log exports.
2FA — Two-Factor Authentication	Authentication requiring two independent credentials — used here: OTP + government ID.
FCM — Firebase Cloud Messaging	Google's cross-platform push notification service, used to alert field workers.
GAN — Generative Adversarial Network	A machine-learning framework in which two neural networks compete, used here to generate aged faces.
GPT-4o	OpenAI's multimodal Generative Pre-trained Transformer, version 4 Omni.
HRFAE — High-Resolution Face Age Editing	A published face-aging model capable of producing high-resolution age-progressed images.
InsightFace	An open-source face-analysis toolkit used for detection and landmarking.

IVFFlat	Inverted File with Flat quantisation — a pgvector index type for fast approximate nearest-neighbour search.
JWT — JSON Web Token	A compact, self-contained token used for authentication and authorisation.
MinIO	An S3-compatible open-source object storage system used for image storage.
NGO — Non-Governmental Organisation	A non-profit organisation working independently of government, e.g., child welfare field workers.
NCRB — National Crime Records Bureau	The Indian government body that compiles national crime statistics including missing person data.
OTP — One-Time Password	A six-digit numeric code, valid for 5 minutes, used for mobile authentication.
PII — Personally Identifiable Information	Any data that can identify an individual directly or indirectly.
pgvector	A PostgreSQL extension providing vector data types and similarity search operators.
RBAC — Role-Based Access Control	A security model where access rights are granted based on a user's role.
SAM — Style-based Age Manipulation	A published face-aging model manipulating age in StyleGAN2 latent space while preserving identity.
TLS — Transport Layer Security	A cryptographic protocol ensuring data privacy and integrity in transit.

"AI suggests, the human decides."

This principle is the foundation of the entire KHOJO system. Every AI-generated result is a recommendation to a human being, never a final verdict.

Every match must be verified physically before a family is informed. Every mistake becomes training data for the next iteration of the model.

— End of Functional Requirements Specification —

KHOJO — Amnex Hackathon 2026 | UC34 | FRS v1.1 | 24 April 2026